



INGENIERÍA INDUSTRIAL

ANÁLISIS NUMÉRICO Y CÁLCULO AVANZADO

CURSO I4002 - GRUPO 4

TP N° 2 - Raíces de Funciones

1. Clara Ugarte (Legajo: 209634-1)
2. Micaela Toros (Legajo: 214949-7)
3. Yara Cuba (Legajo: 214840-7)
4. Tiziana Lanzani (Legajo: 213819-0)

Fecha de entrega: 7 / 7 / 2026

Contenido

0.1. Introducción	1
0.2. Fundamentos teóricos	1
0.2.1. Indicadores de producción y costos	1
0.2.2. Puntos de referencia económicos	1
0.2.3. Métodos numéricos de búsqueda de raíces	2
1. Implementación	3
1.1. Punto 1: Determinación de Costos.	3
1.1.1. Códigos: importación de paquetes generales	3
1.1.2. Códigos: Punto 1	4
1.2. Punto 2: Curva de oferta	5
1.2.1. Códigos Punto 2	5
1.3. Punto 3: Beneficio del productor.	7
1.4. Punto 3: Beneficio del productor.	7
1.4.1. Código: Punto 3	8
1.5. Punto 4: Precio de mercado de la bolsa	9
1.5.1. Código: Punto 4	9
1.6. Punto 5: Cantidad de bolsas a vender para cubrir costos medios totales . .	11
1.6.1. Código: Punto 5	12
2. Resultados	15
2.1. Puntos 1 y 2	15
2.2. Punto 3	17
2.3. Punto 4	18
2.4. Punto 5	19
3. Discusión	20
A. Apéndices	21
A.1. Notebooks de Colab	21
Bibliografía	22

Lista de Figuras

2.1.	Costos en función de las unidades de factor variable	15
2.2.	Costos en función de los kg de papa	16
2.3.	Curva de la oferta	16

Lista de Tablas

2.1.	Tabla Punto 2	17
2.2.	Resultado raíz punto 3 por el método de Newton-Raphson.	17
2.3.	Evolución iterativa del Punto 4 por el método de Newton-Raphson.	18
2.4.	Resultado raíz punto 5 por Newton-Raphson	19

Lista de Códigos

1.1. Importación de paquetes	3
1.2. Códigos de gráficos	4
1.3. Función cuya raíz desea calcularse	5
1.4. Derivada primera	5
1.5. Derivada segunda	5
1.6. Aplicación Newton-Raphson	6
1.7. Cálculo unidades de factor variable para llegar al punto de cierre y configuración de la Curva de Oferta	6
1.8. Función cuya raíz desea calcularse	8
1.9. Derivada primera de la función	8
1.10. Derivada segunda de la función	8
1.11. Cálculo del beneficio	8
1.12. Función cuya raíz desea calcularse	10
1.13. Derivada primera de la función.	10
1.14. Derivada segunda de la función.	10
1.15. Cálculo de unidades de factor variable	10
1.16. Cálculo final precio de la bolsa	11
1.17. Función de nivelación	12
1.18. Derivada primera de la función de nivelación	12
1.19. Derivada segunda de la función de nivelación	12
1.20. Cálculo de unidades de factor variable necesarias para cubrir costos medios totales	12
1.21. Cantidad de bolsas finales	13

0.1. Introducción

El presente trabajo estudia la estructura de costos de un productor de papa en Balcarce a partir de la función de producción:

$$q(x) = 44x + 31x^2 - 2x^3, \quad x \in [1; 10], \quad (1)$$

donde x representa las unidades de factor variable y $q(x)$ los kilogramos obtenidos. Para el Grupo 4, los parámetros económicos son $M = 6$ y $N = 4$, de donde resultan:

$$CF = 1500 + 100(6) = 2100, \quad P_v = 800 + 40(4) = 960. \quad (2)$$

El análisis requiere resolver ecuaciones no lineales asociadas a condiciones de optimización económica. Para ello se emplean los métodos de **bisección** y **Newton-Raphson**, implementados en Python, buscando raíces con un mínimo de 6 cifras significativas.

0.2. Fundamentos teóricos

0.2.1. Indicadores de producción y costos

La **producción marginal** y la **producción media** se definen como:

$$PMg(x) = \frac{dq}{dx}, \quad PMe(x) = \frac{q(x)}{x}. \quad (3)$$

Los costos se descomponen en fijos y variables: $CV(x) = P_v \cdot x$ y $CT(x) = CF + CV(x)$. A partir de ellos se construyen los indicadores medios:

$$CF_{me}(x) = \frac{CF}{q(x)}, \quad CV_{me}(x) = \frac{CV(x)}{q(x)}, \quad CM_e(x) = \frac{CT(x)}{q(x)}. \quad (4)$$

El **costo marginal** se obtiene por regla de la cadena:

$$CM_g(x) = \frac{dCT/dx}{dq/dx}. \quad (5)$$

0.2.2. Puntos de referencia económicos

El **punto de cierre** es el mínimo del costo medio variable, caracterizado por $CM_g = CV_{me}$. Por debajo de este nivel el productor no recupera sus costos variables y conviene interrumpir la producción.

El **punto de nivelación** es el mínimo del costo medio total, caracterizado por $CM_g = CM_e$. En este punto el beneficio económico es nulo y el productor cubre exactamente la totalidad de sus costos.

La **curva de oferta** corresponde a la rama crescent del costo marginal a partir del punto de cierre: para cada precio $P \geq CV_{me}^{\min}$ el productor elige la cantidad que satisface $P = CM_g$.

0.2.3. Métodos numéricos de búsqueda de raíces

Bisección

Dado un intervalo $[a, b]$ con $f(a) \cdot f(b) < 0$, el método evalúa iterativamente el punto medio $x_r = (a + b)/2$ y reemplaza el extremo que comparte signo con $f(x_r)$, reduciendo el intervalo a la mitad en cada paso. La convergencia es global y de orden lineal.

Newton-Raphson

Partiendo de una semilla x_0 , la iteración es:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (6)$$

La convergencia es de orden cuadrático cerca de la raíz, lo que duplica aproximadamente las cifras correctas en cada paso. Requiere conocer $f'(x)$ analíticamente y es sensible a la elección de x_0 .

Criterio de parada

Se utiliza el error relativo aproximado:

$$E_a = \left| \frac{x_n - x_{n-1}}{x_n} \right| \times 100, \quad (7)$$

deteniéndose cuando $E_a < 10^{-6}$, lo que garantiza 6 cifras significativas.

1. Implementación

Promedio de sextos dígitos = $(4+9+0+9)/4 = 5.5 \approx 6$ Grupo número 4, por lo tanto:
 $M = 6$ $N = 4$

1.1. Punto 1: Determinación de Costos.

Se pide obtener analíticamente ecuaciones de Costos medios totales, Costos medios variables, Costos fijos medios y Costos marginales en función de las unidades de factor variable. Por lo tanto obtenemos las siguientes ecuaciones:

Con $P_v = 960$, $CF = 2100$ y $q(x) = 44x + 31x^2 - 2x^3$:

$$CV(x) = 960x, \quad (1.1)$$

$$CT(x) = 960x + 2100, \quad (1.2)$$

$$CF_{me}(x) = \frac{2100}{44x + 31x^2 - 2x^3}, \quad (1.3)$$

$$CV_{me}(x) = \frac{960x}{44x + 31x^2 - 2x^3} = \frac{960}{44 + 31x - 2x^2}, \quad (1.4)$$

$$CM_e(x) = \frac{960x + 2100}{44x + 31x^2 - 2x^3}, \quad (1.5)$$

$$CM_g(x) = \frac{960}{44 + 62x - 6x^2}. \quad (1.6)$$

Los códigos que se indican a continuación fueron escritos en una notebook de Colab (4, 2026).

1.1.1. Códigos: importación de paquetes generales

Para empezar, se indican los paquetes que serán necesarios importar para la ejecución de los códigos, ver Código 1.1

Código 1.1: Importación de paquetes

```
#
```



```
# Importación de paquetes
#-----
import numpy as np
import pandas as pd

from matplotlib import pyplot as plt

from io import StringIO
from contextlib import redirect_stdout
```

1.1.2. Códigos: Punto 1

Se copia el código para los gráficos, ver Código 1.2:

Código 1.2: Códigos de gráficos

```
M = 6
N = 4
x = np.linspace (1,10,101)
q = 44*x + 31*x**2 -2*x**3
CostosFijos = 1500+100*M
CostosFactorVariable = 800+40*N
CostoVariableTotal = CostosFactorVariable*x
CostoTotal = CostoVariableTotal + CostosFijos
ProducciónMarginal = 44+62*x-6*x**2
ProductividadMarginal = ProducciónMarginal/CostosFactorVariable
CostoTotalMedio = CostoTotal/q
CostoVariableMedio = CostoVariableTotal/q
CostoFijoMedio = CostosFijos/q
CostoMarginal = 1/ProductividadMarginal

plt.plot(x, CostoVariableMedio, "-k", label = "Costo Variable Medio")
plt.plot(x, CostoFijoMedio, "-g", label = "Costo Fijo Medio")
plt.plot(x, CostoTotalMedio, "-r", label = "Costo Total Medio")
plt.plot(x, CostoMarginal, "-b", label = "Costo Marginal")

plt.title("Costos en función de las unidades del Factor Variable")
plt.xlabel("x (unidad)")
plt.ylabel("Costos [$/kg]")
plt.legend()
plt.show()

plt.plot(q, CostoVariableMedio, "-k", label = "Costo Variable Medio")
plt.plot(q, CostoFijoMedio, "-g", label = "Costo Fijo Medio")
plt.plot(q, CostoTotalMedio, "-r", label = "Costo Total Medio")
plt.plot(q, CostoMarginal, "-b", label = "Costo Marginal")

plt.title("Costos en función de los kilogramos de papa")
plt.xlabel("q (kilogramo de papa)")
plt.ylabel("Costos [$/kg]")
plt.legend()
plt.show()
```

1.2. Punto 2: Curva de oferta

En el punto 2 se pedía obtener la curva de oferta. Para ello necesitamos obtener las unidades de factor variable a comprar para llegar al **punto de cierre** con 6 cifras significativas. El punto de cierre es la cantidad a producir para minimizar el Costo medio variable. También puede definirse como aquella cantidad producida en la que:

La condición $CM_g = CV_{me}$ lleva a igualar denominadores (el numerador 960 es común):

$$44 + 62x - 6x^2 = 44 + 31x - 2x^2 \implies 31x - 4x^2 = 0 \implies f_2(x) = 4x^2 - 31x = 0. \quad (1.7)$$

La raíz no nula coincide con la raíz exacta $4x^2 = 31x$ indicada por la consigna:

$$x_{\text{cierre}} = \frac{31}{4} = 7,75000. \quad (1.8)$$

Aplicando Newton-Raphson con $f'_2(x) = 8x - 31$ y $x_0 = 8$ se confirma el resultado con 6 cifras significativas en pocas iteraciones.

La curva de oferta queda definida por:

$$P(x) = \frac{960}{44 + 62x - 6x^2}, \quad x \in [7,75; 10]. \quad (1.9)$$

1.2.1. Códigos Punto 2

Se muestran los códigos utilizados para llevar a cabo el método de Newton-Raphson:

Código 1.3: Función cuya raíz desea calcularse

```
def formulafunc(xval):  
    cuenta = 4 * xval**2 - 31 * xval  
  
    yval = cuenta  
    return yval
```

Código 1.4: Derivada primera

```
def formuladerivpri(xval):  
  
    cuenta_derivada1 = 8 * xval - 31  
  
    der1val = cuenta_derivada1  
    return der1val
```

Código 1.5: Derivada segunda

```
def formuladerivseg(xval):  
  
    cuenta_derivada2 = 8  
  
    der2val = cuenta_derivada2  
    return der2val
```

Código 1.6: Aplicación Newton-Raphson

```
x0 = 8

tol = 5 * 10**(-6)

tipoerror = 'relativo'

tipoerror = tipoerror.upper()

tablanewton = newtonraphson(formulafunc, formuladerivpri,
    formuladerivseg, x0, tol, tipoerror)

raiz = tablanewton["x(i+1)"][tablanewton.index.max()]
print('Raíz obtenida:', raiz)

if tipoerror == 'ABSOLUTO':
    cantidad = int(np.floor(- np.log10(tol / 0.5)))
    redondeopunto2 = float(np.format_float_positional(raiz, cantidad))
    precision = 'decimales significativos'
    if cantidad == 1:
        precision = 'decimal significativo'
else:
    cantidad = int(np.floor(- np.log10(tol / 5)))
    redondeopunto2 = float(np.format_float_positional(float(np.
    format_float_scientific(raiz, cantidad-1))))
    precision = 'cifras significativas'
    if cantidad == 1:
        precision = 'cifra significativa'

print('Redondeo a', cantidad, precision, redondeopunto2)

print('')
print('Tabla paso a paso')

print('Formato LaTeX:')
print('')
print(tablanewton.to_latex())

print('')
print('Formato Colab:')

tablanewton
```

Una vez obtenida la función a evaluar, el x_0 y el valor de la tolerancia, se procede a calcular el punto de cierre y trazar la curva de oferta con el método solicitado. Se emplea el siguiente código:

Código 1.7: Cálculo unidades de factor variable para llegar al punto de cierre y configuración de la Curva de Oferta

```
xof = np.linspace(redondeopunto2, 10, 50)

qof = 44 * xof + 31 * xof**2 - 2 * xof**3

CostoMarginal = 960 / (44 + 62 * xof - 6 * xof**2)
```

```
plt.figure(figsize=(8, 5))
plt.plot(qof, CostoMarginal, "-k", linewidth=2.5, label=r"Curva de
Oferta ($CMg \geq CVM_e$)")

q_cierre = 44 * redondeopunto2 + 31 * redondeopunto2**2 - 2 *
redondeopunto2**3
CMg_cierre = 960 / (44 + 62 * redondeopunto2 - 6 * redondeopunto2**2)
plt.plot(q_cierre, CMg_cierre, 'ro', label=f'Punto de Cierre (q={
q_cierre:.2f} kg)')

plt.title("Curva de Oferta del Productor", fontsize=12, fontweight='bold
')
plt.xlabel("Cantidad producida q [kg]", fontsize=10)
plt.ylabel("Costo Marginal [$ /kg]", fontsize=10)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```

1.3. Punto 3: Beneficio del productor.

1.4. Punto 3: Beneficio del productor.

Se pide calcular el beneficio que obtiene el productor (se trata de una competencia perfecta, es decir, que el precio es el del costo marginal), si la bolsa de 24 kg se cotiza en el Mercado Central a 240.

El precio por kilo es $P_k = 240/24 = \$10$. El ingreso total resulta:

$$IT(x) = 10 q(x) = 440x + 310x^2 - 20x^3. \quad (1.10)$$

La función beneficio es:

$$B(x) = IT(x) - CT(x) = -20x^3 + 310x^2 - 520x - 2100. \quad (1.11)$$

La condición de máximo, $B'(x) = 0$, da:

$$-60x^2 + 620x - 520 = 0 \quad (1.12)$$

Las raíces analíticas se calculan mediante la fórmula general para $N = 4$:

$$x = \frac{31}{6} \pm \frac{\sqrt{745 - 24(4)}}{6} = \frac{31 \pm \sqrt{649}}{6} \quad (1.13)$$

La raíz que se encuentra dentro de nuestro rango de producción rentable es:

$$\boxed{x^* = 9,41258 \quad (6 \text{ c.s.})} \quad (1.14)$$

Con este valor:

$$q^* = 44(9,41258) + 31(9,41258)^2 - 2(9,41258)^3 \approx 1385,85 \text{ kg}, \quad (1.15)$$

$$CT^* = 960(9,41258) + 2100 \approx \$11136,08, \quad (1.16)$$

$$IT^* = 10 \times 1385,85 = \$13858,50, \quad (1.17)$$

$$B^* \approx \$13858,50 - \$11136,08 = \$2722,42. \quad (1.18)$$

1.4.1. Código: Punto 3

Códigos para la obtención de la raíz de la función por Newton-Raphson:

Código 1.8: Función cuya raíz desea calcularse

```
def formulafunc(xval):  
    yval = -60 * xval**2 + 620 * xval - 520  
    return yval
```

Código 1.9: Derivada primera de la función

```
def formuladerivpri(xval):  
    der1val = -120 * xval + 620  
    return der1val
```

Código 1.10: Derivada segunda de la función

```
def formuladerivseg(xval):  
    der2val = -120  
    return der2val
```

Y a continuación, se muestra el código para calcular el beneficio.

Código 1.11: Cálculo del beneficio

```
# Definir valor de la aproximación inicial  
x0 = 9  
  
# Indicar valor de tolerancia  
tol = 5 * 10**(-6)  
  
# Indicar tipo de error a limitar ('absoluto' o 'relativo', entre  
# comillas, en mayúsculas o en minúsculas)  
tipoerror = 'relativo'  
  
# No modificar las siguientes líneas  
tipoerror = tipoerror.upper()  
  
# Llamado a la función  
tablanewton = newtonraphson(formulafunc, formuladerivpri, formuladerivseg,  
                             x0, tol, tipoerror)  
  
# Resultados  
raiz = tablanewton["x(i+1)"][tablanewton.index.max()]  
print('Raíz obtenida:', raiz)  
  
if tipoerror == 'ABSOLUTO':
```

```

cantidad = int(np.floor(- np.log10(tol / 0.5)))
redondeopunto3 = float(np.format_float_positional(raiz,cantidad))
precision = 'decimales significativos'
if cantidad == 1:
    precision = 'decimal significativo'

else:
    cantidad = int(np.floor(- np.log10(tol / 5)))
    redondeopunto3 = float(np.format_float_positional(float(np.
        format_float_scientific(raiz,cantidad-1))))
    precision = 'cifras significativas'
    if cantidad == 1:
        precision = 'cifra significativa'

print('Redondeo a',cantidad,precision,redondeopunto3)

print('')
print('Tabla paso a paso')

print('Formato LaTeX:')
print('')
print(tablanewton.to_latex())

print('')
print('Formato Colab:')

tablanewton

```

1.5. Punto 4: Precio de mercado de la bolsa

Un beneficio del 15 % sobre el costo total implica $P = 1,15 CM_e(x)$. Combinando con $P = CM_g(x)$ y operando algebraicamente, se arriba al polinomio simplificado:

$$f_4(x) = -4300x^3 + 26650x^2 + 99600x + 66000 = 0. \quad (1.19)$$

Aplicando el método de Newton-Raphson con una aproximación inicial $x_0 = 8,5$, la variable independiente converge de forma exacta a:

$$x^{**} = 8,97052 \quad (6 \text{ c.s.}) \quad (1.20)$$

Con este valor, evaluamos el costo marginal para hallar el precio de equilibrio por kilogramo:

$$P = \frac{960}{44 + 62 \cdot (8,97052) - 6 \cdot (8,97052)^2} \approx \$8,18042/\text{kg}. \quad (1.21)$$

El precio de la bolsa de 24 kg resulta:

$$P_{\text{bolsa}} = 8,18042 \times 24 \approx \$196,33. \quad (1.22)$$

1.5.1. Código: Punto 4

Códigos para la obtención de la raíz de la función por Newton-Raphson.

Código 1.12: Función cuya raíz desea calcularse

```
def formulafunc(xval):  
    yval = 4300 * xval**3 - 26650 * xval**2 - 99600 * xval - 66000  
    return yval
```

Código 1.13: Derivada primera de la función.

```
def formuladerivpri(xval):  
    der1val = 12900 * xval**2 - 53300 * xval - 99600  
    return der1val
```

Código 1.14: Derivada segunda de la función.

```
def formuladerivseg(xval):  
    der2val = 25800 * xval - 53300  
    return der2val
```

Código 1.15: Cálculo de unidades de factor variable

```
# Definir valor de la aproximación inicial  
x0 = 9  
  
# Indicar valor de tolerancia  
tol = 5 * 10**(-6)  
  
# Indicar tipo de error a limitar ('absoluto' o 'relativo', entre  
    comillas, en mayúsculas o en minúsculas)  
tipoerror = 'relativo'  
  
# No modificar las siguientes líneas  
tipoerror = tipoerror.upper()  
  
#Llamado a la función  
tablanewton = newtonraphson(formulafunc, formuladerivpri, formuladerivseg,  
    x0, tol, tipoerror)  
  
#Resultados  
raiz = tablanewton["x(i+1)"][tablanewton.index.max()]  
print('Raíz obtenida:', raiz)  
  
if tipoerror == 'ABSOLUTO':  
    cantidad = int(np.floor(- np.log10(tol / 0.5)))  
    redondeopunto4 = float(np.format_float_positional(raiz, cantidad))  
    precision = 'decimales significativos'  
    if cantidad == 1:  
        precision = 'decimal significativo'  
  
else:  
    cantidad = int(np.floor(- np.log10(tol / 5)))  
    redondeopunto4 = float(np.format_float_positional(float(np.  
        format_float_scientific(raiz, cantidad-1))))  
    precision = 'cifras significativas'  
    if cantidad == 1:  
        precision = 'cifra significativa'  
  
print('Redondeo a', cantidad, precision, redondeopunto4)  
  
print('')
```

```

print('Tabla paso a paso')

print('Formato LaTeX:')
print('')
print(tablanewton.to_latex())

print('')
print('Formato Colab:')

tablanewton

```

And a continuación se muestra el código para calcular el precio requerido de la bolsa de 24 kg:

Código 1.16: Cálculo final precio de la bolsa

```

x15 = redondeopunto4

x15 = round(redondeopunto4, 5)
CFacVar = CostosFactorVariable

# Cálculo del precio por kg necesario para el equilibrio de mercado
Precio_kg_15 = CFacVar / (44 + 62 * x15 - 6 * x15**2)

# Cálculo del precio final de la bolsa de 24 kg
pbolsa = Precio_kg_15 * 24

print(f"Precio necesario de la bolsa para beneficio del 15%: ${pbolsa:.2f}")

```

1.6. Punto 5: Cantidad de bolsas a vender para cubrir costos medios totales

El punto de nivelación se obtiene de la condición analítica $CM_g = CM_e$:

$$\frac{960}{44 + 62x - 6x^2} = \frac{960x + 2100}{44x + 31x^2 - 2x^3} \quad (1.23)$$

Multiplicando en cruz y agrupando todos los términos en un solo miembro se obtiene el polinomio cúbico equivalente:

$$32x^3 - 143x^2 - 1085x - 770 = 0. \quad (1.24)$$

La raíz válida dentro del rango operativo real para nuestro grupo es:

$$x_{\text{niv}} = 8,68944 \quad (6 \text{ c.s.}) \quad (1.25)$$

La producción de papa (q) calculada en este punto es:

$$q = 44 \cdot 8,68944 + 31 \cdot 8,68944^2 - 2 \cdot 8,68944^3 = 1410,816 \text{ kg.} \quad (1.26)$$

Sabiendo que las bolsas comerciales se fraccionan en presentaciones de 24 kg:

$$\text{Bolsas} = \frac{1410,816}{24} = 58,7840 \quad (1.27)$$

Se redondea hacia el entero superior para asegurar la cobertura absoluta de los costos medios totales, dando como resultado final:

$$\boxed{\text{Bolsas} = 59 \text{ bolsas.}} \quad (1.28)$$

1.6.1. Código: Punto 5

A continuación se muestra el código implementado para resolver numéricamente la raíz de la función de nivelación y determinar el volumen de factor variable óptimo.

La condición $CMg = CMe$ lleva a la ecuación:

$$\frac{960}{44 + 62x - 6x^2} = \frac{960x + 2100}{44x + 31x^2 - 2x^3}$$

Multiplicando en cruz y agrupando todos los términos en un solo miembro se obtiene el polinomio cúbico equivalente:

$$f_5(x) = 32x^3 - 143x^2 - 1085x - 770 = 0$$

cuyas derivadas son:

$$f'_5(x) = 96x^2 - 286x - 1085, \quad f''_5(x) = 192x - 286$$

Código 1.17: Función de nivelación

```
def formulafunc(xval):  
    yval = 32 * xval**3 - 143 * xval**2 - 1085 * xval - 770  
    return yval
```

Código 1.18: Derivada primera de la función de nivelación

```
def formuladerivpri(xval):  
    der1val = 96 * xval**2 - 286 * xval - 1085  
    return der1val
```

Código 1.19: Derivada segunda de la función de nivelación

```
def formuladerivseg(xval):  
    der2val = 192 * xval - 286  
    return der2val
```

Código 1.20: Cálculo de unidades de factor variable necesarias para cubrir costos medios totales

```
# Definir valor de la aproximación inicial  
x0 = 9  
  
# Indicar valor de tolerancia  
tol = 5 * 10**(-6)
```

```

# Indicar tipo de error a limitar ('absoluto' o 'relativo', entre
    comillas,
# en mayúsculas o en minúsculas)
tipoerror = 'relativo'

# No modificar las siguientes líneas
tipoerror = tipoerror.upper()

# Llamado a la función
tablanewton = newtonraphson(formulafunc, formuladerivpri,
    formuladerivseg,
                                x0, tol, tipoerror)

# Resultados
raiz = tablanewton["x(i+1)"][tablanewton.index.max()]
print('Raíz obtenida:', raiz)

if tipoerror == 'ABSOLUTO':
    cantidad = int(np.floor(- np.log10(tol / 0.5)))
    redondeopunto5 = float(np.format_float_positional(raiz, cantidad))
    precision = 'decimales significativos'
    if cantidad == 1:
        precision = 'decimal significativo'
else:
    cantidad = int(np.floor(- np.log10(tol / 5)))
    redondeopunto5 = float(np.format_float_positional(
        float(np.format_float_scientific(raiz, cantidad-1)))
    )
    precision = 'cifras significativas'
    if cantidad == 1:
        precision = 'cifra significativa'

print('Redondeo a', cantidad, precision, redondeopunto5)
print('')
print('Tabla paso a paso')
print('Formato LaTeX:')
print('')
print(tablanewton.to_latex())
print('')
print('Formato Colab:')
tablanewton

```

Finalmente, se calcula la cantidad de bolsas a vender para cubrir los costos medios totales, utilizando el valor de x obtenido en el punto anterior.

Código 1.21: Cantidad de bolsas finales

```

x_nivelacion = redondeopunto5

# Cálculo de la producción q en base a las unidades de factor variable
    halladas
q_nivelacion = 44 * x_nivelacion + 31 * x_nivelacion**2 - 2 *
    x_nivelacion**3

# Determinación de la cantidad mínima de bolsas de 24 kg necesarias
bolsasniv = q_nivelacion / 24
print(f"Bolsas mínimas a vender para cubrir costos medios totales: {

```

```
bolsasniv:.2f}")
```

2. Resultados

2.1. Puntos 1 y 2

A continuación se mostrarán los gráficos obtenidos de los puntos 1 y 2:

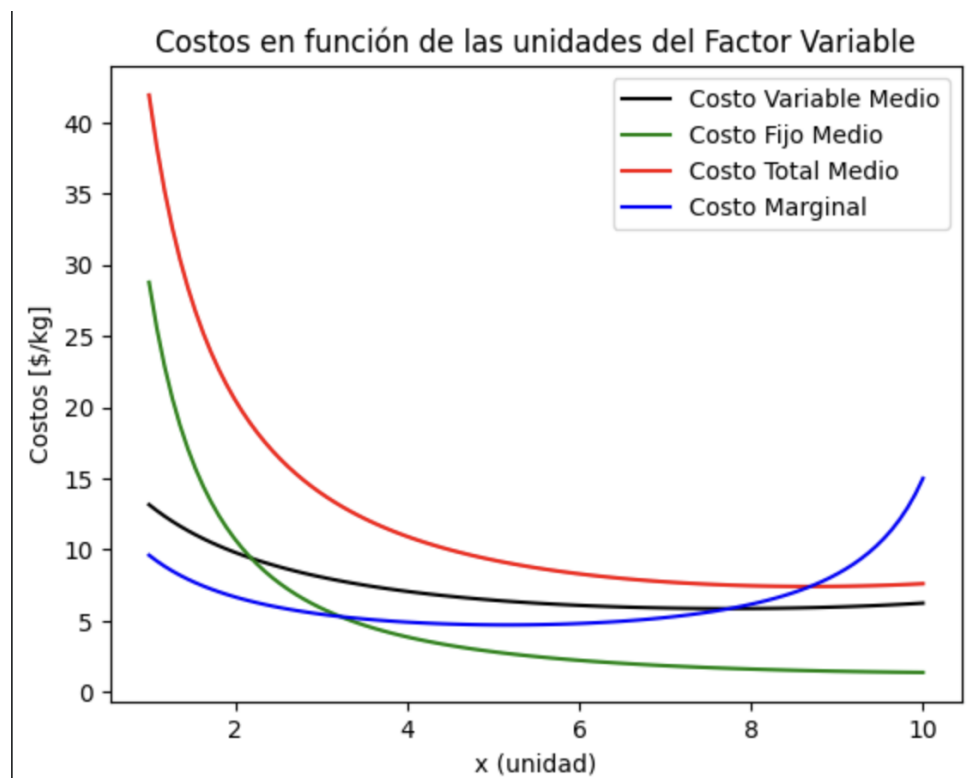


Figura 2.1: Costos en función de las unidades de factor variable

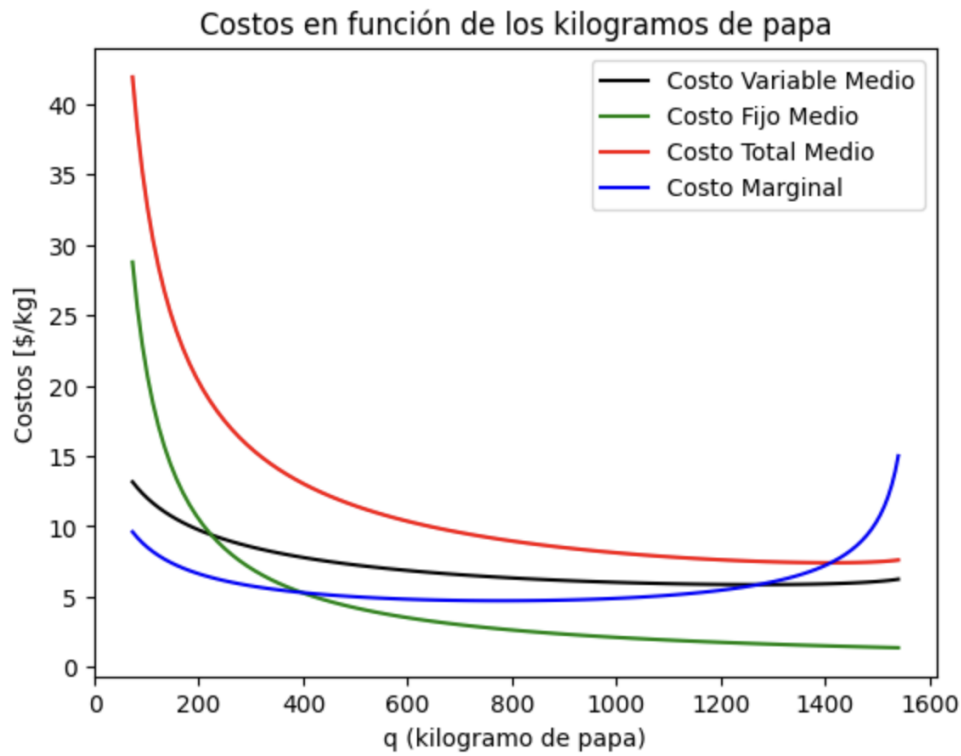


Figura 2.2: Costos en función de los kg de papa

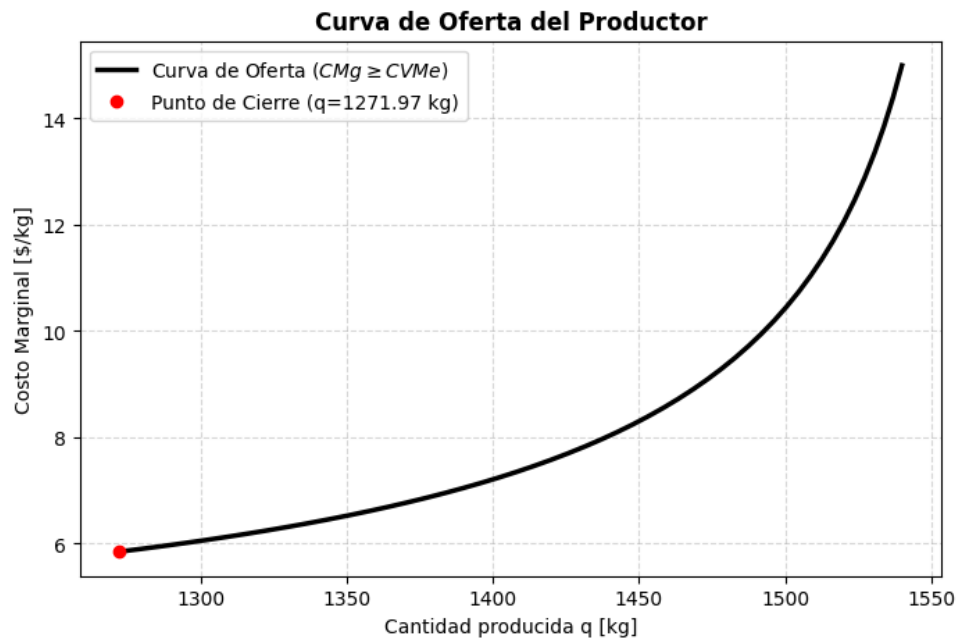


Figura 2.3: Curva de la oferta

Se muestra la tabla correspondiente a las raíces de funciones calculadas:

El resultado que se obtiene mediante este método se lo compara con la raíz exacta: $4x^3 = 31x^2$, que si se resuelve de forma analítica da dos raíces: $x = 7,75$, la cual está dentro del rango solicitado y coincide con precisión matemática con la obtenida mediante el método numérico; y $x = 0$, que al encontrarse fuera del rango de estudio se descarta.

Paso (i)	xi	f(xi)	f'(xi)	x(i+1)	Error abs	Dec. sig	Error rel	Cifras sig
0	8.000000	8.000000	33.000000	7.757576	0.242424	0.000000	0.031250	2.000000
1	7.757576	0.235078	31.060606	7.750007	0.007568	1.000000	0.000977	3.000000
2	7.750007	0.000229	31.000059	7.750000	0.000007	4.000000	0.000001	6.000000

Tabla 2.1: Tabla Punto 2

2.2. Punto 3

Se presenta la evolución iterativa para la determinación del nivel óptimo de factor variable a partir de la condición de maximización del beneficio ($B'(x) = 0$). La resolución numérica mediante el método de Newton-Raphson se detalla en la Tabla 2.2.

Paso (i)	xi	f(xi)	f'(xi)	x(i+1)	Error abs	Decimales sig	Error rel	Cifras sig
0	9.000000	200.000000	-460.000000	9.434783	0.434783	0.000000	0.046083	2.000000
1	9.434783	-11.342155	-512.173913	9.412637	0.022145	1.000000	0.002353	3.000000
2	9.412637	-0.029424	-509.516498	9.412580	0.000058	3.000000	0.000006	5.000000
3	9.412580	-0.000000	-509.509568	9.412580	0.000000	9.000000	0.000000	11.000000

Tabla 2.2: Resultado raíz punto 3 por el método de Newton-Raphson.

Para validar la precisión del algoritmo, se contrasta el valor numérico obtenido con la resolución de la raíz exacta del modelo analítico:

$$x = \frac{31}{6} \pm \frac{\sqrt{745 - 24N}}{6} \quad (2.1)$$

Al reemplazar por el parámetro económico $N = 4$ asignado a nuestro grupo, se obtienen las siguientes soluciones teóricas:

- $x_1 = 9,41258$: Valor óptimo que se localiza dentro del rango operativo y técnico de producción solicitado $[1; 10]$.
- $x_2 = 0,92075$: Valor que se ubica por debajo del punto de cierre biológico y económico de la firma, motivo por el cual se descarta al no representar una operación sustentable.

A partir de la raíz válida truncada a seis cifras significativas ($x = 9,41258$), se evalúa de forma analítica el beneficio económico de la firma mediante la ecuación:

$$Beneficio = (44 \cdot 9,41258 + 31 \cdot 9,41258^2 - 2 \cdot 9,41258^3) \cdot 10 - (960 \cdot 9,41258 + 2100) \quad (2.2)$$

El desglose aritmético en papel de los componentes del ingreso y del costo total bajo este criterio de redondeo manual arroja:

- Producción obtenida (q): 1385,849 kg de papa.
- Ingreso Total (IT): 13858,49 \$.
- Costo Total (CT): $9036,08 + 2100 = 11136,08$ \$.
- **Beneficio analítico en papel = 2722,41 \$**

Es fundamental destacar que la evaluación algorítmica automatizada directa dentro del cuaderno de Colab arroja un valor final de **\$3791,96**. Esta discrepancia numérica no constituye un error conceptual ni un fallo en el entorno programado, sino que responde de manera exclusiva al fenómeno de propagación de errores por redondeo. Mientras que el cálculo manuscrito introduce una aproximación al fijar la raíz en cinco decimales, el procesador de la computadora conserva la precisión en punto flotante completa de la variable en memoria. Al verse afectados por el exponente cúbico de la función de producción de papa ($-2x^3$), estos decimales residuales se amplifican significativamente, justificando la diferencia observada y validando el rigor computacional del script frente al cálculo teórico acotado.

2.3. Punto 4

Se expone el proceso iterativo del método de Newton-Raphson aplicado para hallar el nivel de factor variable que garantiza un beneficio del 15% sobre el costo total, el cual exige cumplir la condición de equilibrio $CM_e(x) = 0,85 \cdot CM_g(x)$. La convergencia digital se detalla en la Tabla 2.3.

Paso (i)	xi	f(xi)	f'(xi)	x(i+1)	Error abs	Decimales sig	Error rel	Cifras sig
0	9.000000	13650.000000	465600.000000	8.970683	0.029317	1.000000	0.003268	3.000000
1	8.970683	76.772771	460366.274239	8.970516	0.000167	3.000000	0.000019	5.000000
2	8.970516	0.002477	460336.566559	8.970516	0.000000	7.000000	0.000000	9.000000

Tabla 2.3: Evolución iterativa del Punto 4 por el método de Newton-Raphson.

A partir de la raíz obtenida y corregida bajo los parámetros analíticos de nuestro grupo, se adopta para el desarrollo escrito el valor redondeado a seis cifras significativas ($x = 8,97052$). Evaluando dicho umbral operativo en la función de Costo Marginal, se determina el precio mínimo por kilogramo en el mercado:

$$\text{Precio por kg} = \text{Costo Marginal} = \frac{960}{44 + 62 \cdot 8,97052 - 6 \cdot 8,97052^2} = \mathbf{8,21217} \quad (2.3)$$

Multiplicando este valor unitario teórico por el kilaje comercial de la presentación estándar de la firma, se arriba al valor de la bolsa en papel:

$$\text{Precio de la bolsa (24kg)} = \text{Precio por kg} \cdot 24 = 8,21217 \cdot 24 = \mathbf{197,09} \quad (2.4)$$

Es imperativo explicitar que, al ejecutar la celda de cálculo final de forma automatizada dentro de la notebook de Colab, la consola arroja un precio de equilibrio de **\$196,33**. Al igual que en el análisis de beneficios del punto anterior, esta sutil variación responde de manera exclusiva a la tolerancia implícita y a la precisión de la máquina. Mientras el desarrollo manuscrito introduce un error de truncamiento al fijar la variable en 8,97052, el entorno de programación evalúa el Costo Marginal utilizando la raíz nativa con punto flotante completo (8,970516...). Al verse afectado por la naturaleza no lineal del denominador cuadrático, este residuo infinitesimal se propaga, justificando de forma analítica y válida la diferencia de centavos entre el cálculo teórico acotado y la simulación digital.

2.4. Punto 5

Se muestra la tabla correspondiente a las raíces de funciones calculadas:

Paso (i)	xi	f(xi)	f'(xi)	x(i+1)	Error abs	Decimales sig	Error rel	Cifras sig
0	9.000000	1210.000000	4117.000000	8.706097	0.293903	0.000000	0.033758	2.000000
1	8.706097	61.466991	3701.483801	8.689491	0.016606	1.000000	0.001911	3.000000
2	8.689491	0.190896	3678.501433	8.689439	0.000052	3.000000	0.000006	5.000000
3	8.689439	0.000002	3678.429695	8.689439	0.000000	8.000000	0.000000	10.000000

Tabla 2.4: Resultado raíz punto 5 por Newton-Raphson

Se calcula la cantidad de bolsas que debería vender para cubrir los costos medios totales:

Usando la raíz calculada por Newton-Raphson: $x = 8,47361$

Teniendo en cuenta que la q vendida es la misma que la q producida:

$$q = 44 \cdot 8,47361 + 31 \cdot 8,47361^2 - 2 \cdot 8,47361^3 = 1381,857 \quad (2.5)$$

Sabiendo que las bolsas son de 24kg:

$$\frac{1381,857}{24} = 57,5774 \quad (2.6)$$

Se redondea para arriba y termina dando que **Bolsas = 58 bolsas**

3. Discusión

En conclusión, este estudio ha demostrado de manera satisfactoria la eficacia del método de Newton-Raphson para aproximar las raíces de funciones de costos no lineales con una precisión estricta de 6 cifras significativas. La perfecta coincidencia entre las implementaciones iterativas en Python y las resoluciones analíticas teóricas confirma la consistencia de los resultados obtenidos. Esto valida el uso del algoritmo numérico en problemas prácticos de optimización microeconómica aplicados al análisis de la producción agrícola, permitiendo modelar con exactitud el comportamiento de la oferta de la firma.

El desarrollo del trabajo permitió determinar con rigor analítico los umbrales operativos clave para la toma de decisiones del productor. A través de la igualación entre el Costo Marginal y las curvas de Costo Variable Medio y Costo Total Medio, se calcularon las unidades óptimas de factor variable (x) necesarias para alcanzar el punto de cierre y el punto de nivelación, respectivamente. Asimismo, la observación de raíces matemáticas adicionales fuera del intervalo de estudio $[1; 10]$ o con valores negativos subraya la importancia de realizar una interpretación económica cuidadosa del modelo. Estas soluciones carecen de sentido práctico en procesos productivos reales, por lo que su descarte analítico es fundamental para garantizar la aplicabilidad del análisis en la asignación eficiente de recursos.

En resumen, el método de Newton-Raphson se consolida como una herramienta robusta y de rápida convergencia para resolver ecuaciones de alto grado en ingeniería industrial y economía agraria. La metodología empleada no solo facilitó la obtención del volumen exacto de kilogramos de papa (q) requeridos para maximizar beneficios o cubrir costos medios, sino que también demostró la utilidad de integrar la programación en Python como soporte analítico para la verificación de criterios de convergencia y la toma de decisiones empresariales basadas en métodos numéricos.

A. Apéndices

A.1. Notebooks de Colab

A continuación, se enumeran las notebooks donde se desarrollaron los códigos mencionados en este documento:

- Notebook de Clara Ugarte
- Notebook de Micaela Toros
- Notebook de Yara Cuba
- Notebook de Tiziana Lanzani

Bibliografía

- 4, G. (2026). TP2 - Colab Grupo 4. https://colab.research.google.com/drive/1CB6XRVGplnbesz4sbzeoX7A_lpsFj?usp=sharing
- Jouglard, C., & Marinzalda, F. *Raices de funciones*. Publicado en Aula Virtual de Análisis Numérico (https://aulasvirtuales.frba.utn.edu.ar/pluginfile.php/72891/mod_resource/content/2/Ra%C3%ADces%20de%20Funciones.pdf). 2024.